Vapor Stone
RAIL SYSTEMS
A Wabtec company

10 655 Henri-Bourassa Blvd., West
Saint-Laurent (Québec) Canada
H4S 1A1

Tel: 514.335.4200
Fax: 514.335.4231

www.wabtec.com

Regional Transportation District

RTD Denver EMU

Project # 7230

Door System

Software Design Description (SDD)
Input / Output (IO) Module

Contract Number	-	CDRL Number	-
		Customer Ref. Number	TBD

NOTICE
Plans, specifications and engineering data submitted herewith or which may be submitted, remain the property of **Vapor Stone Rail Systems** and shall not be reproduced or altered or used in whole or in part except by express consent of **VSRS**.

AVIS
Les plans, spécifications et données techniques soumis ci-joints, ou qui peuvent être soumis, restent la propriété de **Vapor Stone Rail Systems** et ne doivent pas être reproduits, altérés ou utilisés, totalement ou partiellement, sans le consentement express de **VSRS**.

	Name / Title	Signature / Date
Prepared :	James Kim Software Designer	James Kim 2011-09-26
Verified :	Martin Ouellet, Eng. Software Department Supervisor	2011-09-27
Approved :	Aldo Di Rocco Project Manager	For Aldo De Rocco 2011-09-27

Document No.	**201111628**	Revision	**1**
		Date (yyyy-mm-dd)	**2011-09-26**

Revision List

Rev.	Date	By	ECD	Description of changes
1	2011-09-26	James Kim	---	Official release
0.1	2011-09-23	James Kim	---	Implementing comments from internal inspection I-1834
0	2011-09-14	James Kim	---	Original issue

Preface

This Software Design Description (SDD) complies with the requirements of the IEEE 1016-1998 and IEEE 1558-2004.

Table of Contents

REVISION LIST **2**

PREFACE **2**

TABLE OF CONTENTS **3**

LIST OF FIGURES **4**

LIST OF TABLES **4**

1 INTRODUCTION **5**

1.1 Purpose 5
1.2 Scope 5
1.3 Definitions and Acronyms 5
1.3.1 Definitions 5
1.3.2 Acronyms 5
1.4 Document Organization 6

2 REFERENCES **7**

3 SOFTWARE DESIGN **8**

3.1 Decomposition Description 8
3.1.1 Module Decomposition 8
3.1.2 Concurrent Process Decomposition 8
3.1.3 Data Decomposition 9
3.2 Dependency Description 11
3.2.1 Inter-Module Dependencies 11
3.2.2 Inter-Process Dependencies 11
3.2.3 Data Dependencies 11
3.3 Interface Description 12
3.3.1 Module Interfaces 12
3.4 Detailed Design 15
3.4.1 IO Module Detailed Design 15
3.4.2 Data Entities Detailed Design 20

List of Figures

Figure 3-1: Inter-Module Dependencies Diagram 11
Figure 3-2: Inter-Process Dependencies Diagram 11
Figure 3-3: IO_Task Flowchart 15
Figure 3-4: io_Read Flowchart 16
Figure 3-5: io_Debounce Flowchart 17

List of Tables

Table 3-1: Structure of IO Signals In 9
Table 3-2: Structure of CPU Signals Out 10
Table 3-3: Motor Voltage Look Up Table 19

1 INTRODUCTION

This document is the Software Design Description of the Input / Output (IO) module developed by Vapor Stone Rail Systems.

The IO Module is a part of the Door Control Unit (DCU) Application that manages the input and output of the controller.

1.1 Purpose

The purpose of this Software Design Description (SDD) is to describe the software components and required implementation of the IO Module.
The intended audience is the Vapor Stone Rail Systems software development team and maintenance team members of the transit authority and the car builder.

1.2 Scope

The IO Module is a software module used for reading inputs and activating outputs of a controller; therefore, it is safety related module.
This module interfaces with other modules.

1.3 Definitions and Acronyms

1.3.1 Definitions

Door Control Unit: A specific processor based electronic module running a Door Control Unit Application. It is used for controlling and monitoring the door motor.

1.3.2 Acronyms

Acronym	Meaning
ASCII	American Standard Code for Information Interchange
CDRL	Contract Data Requirements List
CL	Car Line
CPLD	Complex Programmable Logic Device
CPU	Central Processing Unit
DCU	Door Control Unit
DSR	Door Summary Relay
ECD	Engineering Change Document
FLOG	Fault Logger
IEEE	Institute of Electrical and Electronics Engineers
IO	Input and Output
LH	Left Hand
LSS	Lock Sensing Switch
MHz	MegaHertz
MONIT	Monitoring
ms	Millisecond
NVRAM	Non-Volatile Random Access Memory
OC	Over Current
OS	Operating System
PID	Proportional Integral Derivative
PSS	Panel Sensing Switch
PT	Portable Test
PWM	Pulse Width Modulation
PWR	Power
RAM	Random Access Memory
RH	Right Hand
RTD	Regional Transportation District
RTDM	Real Time Data Monitoring

Acronym	Meaning
RTN	Return
SCI	Software Configuration Items
SDD	Software Design Description
SRAM	Static Random Access Memory
SRS	Software Requirements Specification
Std	Standard
TL	Train Line
VSRS	Vapor Stone Rail Systems
W/O	Without

1.4 Document Organization

This SDD is divided into three sections and the following specific information is provided within each one:

- Section 1, Introduction, states the purpose, identifies the scope, gives definitions, abbreviations and an overview of the SDD.
- Section 2, References, lists the factors that affect the product and its requirements.
- Section 3, Software Design, summarizes followings:
    - Decomposition Description: describes functions of software entities.
    - Dependency Description: gives a brief description of the dependency between each entity.
    - Interface Description: describes the interface of the entity.
    - Detailed Design: details the functions implemented by each software entity, presents and explains tasks performed by each software entity in functional conditions.

Within the document, diagrams, flowcharts and tables are used to present the software design:

- Decision flowcharts give a graphical view of how entities and functions process work.
- The diagram Figure 3-1 describes the dependency between different modules.
- Tables contain data structures used by the IO Module. They also present C code.

2 REFERENCES

This section lists the number, title, revision, date, and source of all documents referenced in this SDD.

Standards

IEEE Std 1016-1998	IEEE Recommended Practice for Software Design Descriptions, September 23rd 1998, IEEE, www.ieee.org
IEEE Std 1558-2004	IEEE Recommended Practice for Software Documentation for Rail Equipment and Systems, March 23rd 2005, IEEE, www.ieee.org

Vapor Stone Rail Systems Documents

201111208	Software Quality Assurance Plan, Door Control System, Rev. 0
201111270	Software Requirements Specification, Door Control System - DCM Application, Rev. 1
201111576	Software Application Configuration Specification, Rev. 1
S17-07	Software Design Description Template, Rev. 0, 2002-01-15[1]

[1] Complemented with IEEE STD 1558 Requirements

3 SOFTWARE DESIGN

3.1 Decomposition Description

3.1.1 Module Decomposition

3.1.1.1 IO Module Description

The IO Module is a single entity module performing following functions:

- Read inputs and filter out the unstable signal
- Activate the output as commanded
- Verify the mismatch and feedback signals
- Set the direction of the H-bridge motor circuit according to the command

3.1.2 Concurrent Process Decomposition

3.1.2.1 IO Task Description

The IO task loop runs every 50ms and calls the internal functions. This task is running as an independent thread.

3.1.3 Data Decomposition

3.1.3.1 IO Data Description

The IO Module uses the following structure to read inputs.

Table 3-1: Structure of IO Signals In

Identification	SignalsIn

Prototype	typedef struct _SignalsIn
	{
	union _Loc_Inputs
	{
	unsigned long ulLocal_Inputs; // local inputs
	struct _Locals_Inputs
	{
	unsigned char LSS_Right :1; // Lock Sensing Switch Right
	unsigned char LSS_Left :1; // Lock Sensing Switch Left
	unsigned char PSS_Right :1; // Panel Sensing Switch Right
	unsigned char PSS_Left :1; // Panel Sensing Switch Left
	unsigned char ZeroSpeed_PWR_CL :1; // Zero Speed (+) CL
	unsigned char ZeroSpeed_RTN_CL :1; // Zero Speed (-) CL
	unsigned char DOpen_PWR_TL :1; // Door Open (+) TL
	unsigned char DOpen_RTN_TL :1; // Door Open (-) TL
	unsigned char PDEnable_PWR_TL :1; // Passenger Door Enable (+) TL
	unsigned char PDEnable_RTN_TL :1; // Passenger Door Enable (-) TL
	unsigned char DClose_TL :1; // Close Train line
	unsigned char CS_DOpen :1; // Crew Switch Door Open PWR
	unsigned char CS_DEnable :1; // Crew Switch Door Open RTN
	unsigned char CS_DClose :1; // Crew Switch Door Close
	unsigned char DCMech_Lock_Normal :1; // Door Cut-out Mechanical Lock - Normal
	unsigned char Emergency_Handle :1; // Emergency Handle Switch.
	unsigned char Spare_Input :1; // Spare Input
	unsigned char CPLD_Serial_Data :1; // CPLD Serial Data
	unsigned char CPLD_Open_Interlock :1; // CPLD Open Interlock.
	unsigned char DSR_High :1; // DSR High Side Monitoring
	unsigned char DSR_Low :1; // DSR Low Side Monitoring
	unsigned char unused :1; // unused bit for padding.
	} Apart;
	} Loc_Inputs;
	
	union _FeedBack_Inputs
	{
	unsigned int uiFeedBackInputs; // feedback for the activation of the outputs
	struct _Apart_FeedBack_Inputs
	{
	unsigned char Encoder_Phase_A :1; // Motor Position Encoder phase A
	unsigned char Encoder_Phase_B :1; // Motor Position Encoder phase B
	unsigned char Encoder_Index :1; // Motor encoder index
	unsigned char Door_Open_Supply :1; // Open Power.
	unsigned char SLock_High_Supply :1; // Solenoid High Side
	unsigned char SLock_Low_Control :1; // Solenoid Low side
	unsigned char OC_High_Limit :1; // Pulse for the duration of the over current high limit
	unsigned char OC_Normal_Limit :1; // Pulse for the duration of the over current normal limit
	unsigned char DC_Visual_Alarm :1; // Door Closing Visual Alarm.
	unsigned char DC_Audible_Warning :1; // Door Closing Audible Warning.
	unsigned char Spare_Output :1; // Spare output.
	unsigned char OC_Shutdown :1; // Over Current Shutdown.
	} Apart;
	} FeedBack;
	
	union _DiagIn
	{
	unsigned int uiDiagnostic_Inputs; // Voltage supply monitoring
	struct _Apart_Diag_Inputs
	{
	unsigned char Supply_5V_Logic :1; // supply voltage of the logic/CPU is within boundaries.
	unsigned char Supply_5V_Encoder :1; // supply voltage of the Encoder circuit is within boundaries.
	unsigned char Supply_15V_Motor :1; // supply voltage of the motor drive is within boundaries.
	unsigned char Supply_V_Gate_Open :1; // supply voltage of the H-Bridge Open is within boundaries.
	unsigned char Supply_V_Gate_Close :1; // supply voltage of the H-Bridge Close is within boundaries.

```
            unsigned char unused                    :1;     // unused bit for padding.
        } Apart;
    } DiagIn;

    int iMotor_Power_Voltage;           // The motor power input voltage
    int iMotor_Current[10];             // The current read from motor drive.
    long lMotor_Current_Sum;            // The motor current summation
    int iDoorPosition;                  // The current percentage of door open position
    int iDCU_Temperature;               // The DMCU PCB Temperature.
    int i2_5VReference;                 // The 2.5V analog reference, to validate the 4.5V one.

} SignalsIn;
```

The IO Module uses the following structure to activate outputs:

Table 3-2: Structure of CPU Signals Out

Identification	CPU_SignalsOut

Prototype	typedef struct _CPU_SignalsOut
	{
	unsigned char Test_Open_Mot_Out; // Open motor out test
	unsigned char Test_Close_Mot_Out; // Close motor out test
	unsigned char Test_Mode_Mot_Out // Test mode motor
	unsigned char Cpu_Com_Out; // CPU communication
	unsigned char NorCurLim_Act_Out; // Normal current limit
	unsigned char RS232_RTS_Out; // RS232 RTS
	unsigned char Dir_Motor_Open_Iso_Out; // Direction motor open
	unsigned char Dir_Motor_Close_Iso_Out; // Direction motor close
	unsigned char CPU_Open_Interlock; // CPU Open Interlock
	unsigned char DC_Visual_Alarm; // Door closing visual alarm
	unsigned char DC_Audible_Warning; // Door closing audible warning
	unsigned char Cpu_Led_Test_Out; // LED Test
	unsigned char Cpu_Normal_Led_Out; // LED Normal
	unsigned char Cpu_Warning_Led_Out; // LED Warning
	unsigned char Cpu_Fault_Led_Out; // LED Fault
	unsigned char PWM_Motor_Out; // PWM motor
	unsigned char PWM_Solenoid_Act_Out; // PWM solenoid
	unsigned char Spare_Act_Out; // Spare output
	}CPU_SignalsOut;

3.2 Dependency Description

3.2.1 Inter-Module Dependencies

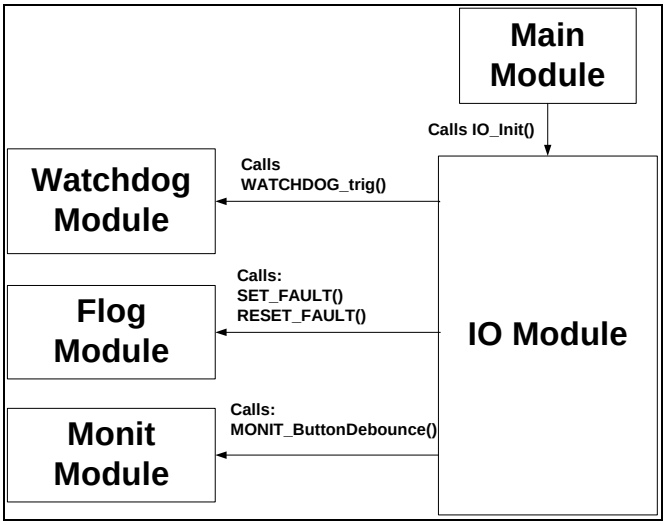

Figure 3-1: Inter-Module Dependencies Diagram

3.2.1.1 IO Module

Figure 3-1 shows the Inter-Module Dependencies Diagram of the IO Module. It displays IO Module's dependency over other modules.

3.2.2 Inter-Process Dependencies

Figure 3-2 displays the two semaphores IO_Task sends to other modules:

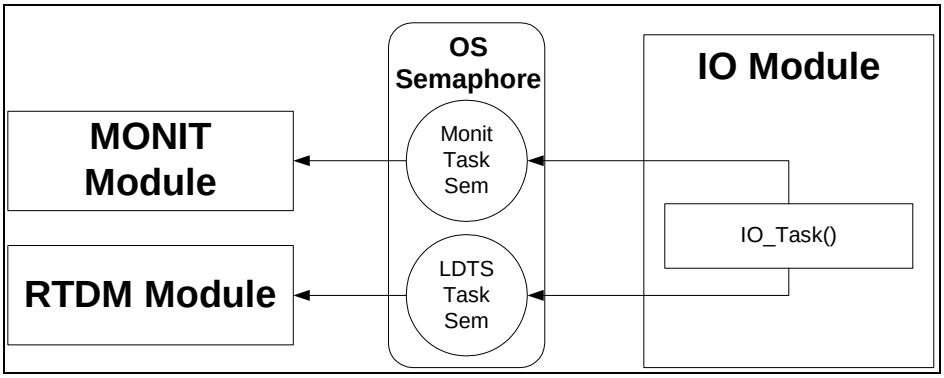

Figure 3-2: Inter-Process Dependencies Diagram

3.2.3 Data Dependencies

Not applicable.

3.3 Interface Description

This section describes the functions of IO Module, listed in alphabetical order.

3.3.1 Module Interfaces

3.3.1.1 IO Module External Interfaces

Function Name	void IO_CurrentLevel (unsigned char ucCommand)
Description	This function selects the high, normal or low current level
Input	ucCommand: It could be CURRENT_HIGH or CURRENT_NORMAL
Output	None.
Return	None.

Function Name	void IO_Init(void)
Description	This function initializes the input and output of CPU, and it initializes the variables.
Input	None.
Output	None.
Return	None.

Function Name	char IO_Motor_Direction (unsigned char ucCommand)
Description	This function sets up the direction of the H-Bridge.
Input	ucCommand: It could be PID_DIR_REQUEST, PID_DIR_OPENING, PID_DIR_BRAKE_HIGH, PID_DIR_CLOSING, PID_DIR_NEUTRAL, PID_DIR_BRAKE_LOW, PID_DIR_TEST_OPEN_HIGH, PID_DIR_TEST_CLOSE_HIGH, PID_DIR_TEST_OPEN_LOW, and PID_DIR_TEST_CLOSE_LOW.
Output	None.
Return	Current direction of the bridge

Function Name	void IO_Task (void)
Description	This task function is scheduled every 50ms.
Input	None.
Output	None.
Return	None.

Function Name	void IO_ReadCurrent(void)
Description	This function reads the Motor Current from an analog input.
Input	None.
Output	None.
Return	None.

3.3.1.2 IO Module Internal Interface

Function Name	void io_Activate_Outputs (void)
Description	This function activates/deactivates outputs according to their state and the flashing time base. It uses the global variable as seen in Table 3-2.
Input	None.
Output	None.
Return	None.

Function Name	void io_CheckFeedback(void)
Description	This function checks feedbacks of outputs. It uses the global variable as seen in Table 3-1.
Input	None.
Output	None.
Return	None.

Function Name	void io_CheckMismatch(void)
Description	This function checks discrepancies on redundant inputs. It uses the global variable as seen in Table 3-1.
Input	None.
Output	None.
Return	None.

Function Name	void io_CheckSensors(void)
Description	This function checks the Panel Sensing Switches and Lock Sensing Switches. It also verifies the Panel Summary Status. It uses the global variable as seen in Table 3-1.
Input	None.
Output	None.
Return	None.

Function Name	void io_CheckStuck(void)
Description	This function checks the signals that have possibly be stuck high. It uses the global variable as seen in Table 3-1.
Input	None.
Output	None.
Return	None.

Function Name	void io_Debounce (void)
Description	This function debounces inputs. It uses the global variable as seen in Table 3-1.
Input	None.
Output	None.
Return	None.

Function Name	void io_Read(void)
Description	This function reads CPU inputs 3 times and stores them temporary for the debouncing. The structure used for temporary storing is as seen in Table 3-1.
Input	None.
Output	None.
Return	None.

Function Name	void io_ReadCPLDVersion(void)
Description	This function reads CPLD version using CPLD serial data.
Input	None.
Output	None.
Return	None.

Function Name	void io_ReadDCUTemp(void)
Description	This function reads analog conversion for the DCU temperature.
Input	None.
Output	None.
Return	None.

Function Name	void io_ReadMotorVoltage(void)
Description	This function reads the Motor Voltage using the Timer 5 of the CPU.
Input	None.
Output	None.
Return	None.

3.4 Detailed Design

This section details all operations performed by IO Module.

3.4.1 IO Module Detailed Design

The following sub-sections describe each function.

3.4.1.1 IO_Init Function

This function first sets the direction of the CPU ports, and it initializes all inputs and outputs of the CPU. Then it initializes the variables and creates the IO task.

3.4.1.2 IO_Task Function

This function reads and validates the inputs each 50 ms, see Figure 3-3. Any discrepancy found is reported to the Monitoring module. The following sections indicate how the verification and validation are performed for each input. Those inputs are registers read on the CPU input/outputs pin ports, and 16 bits on the CPU data bus.

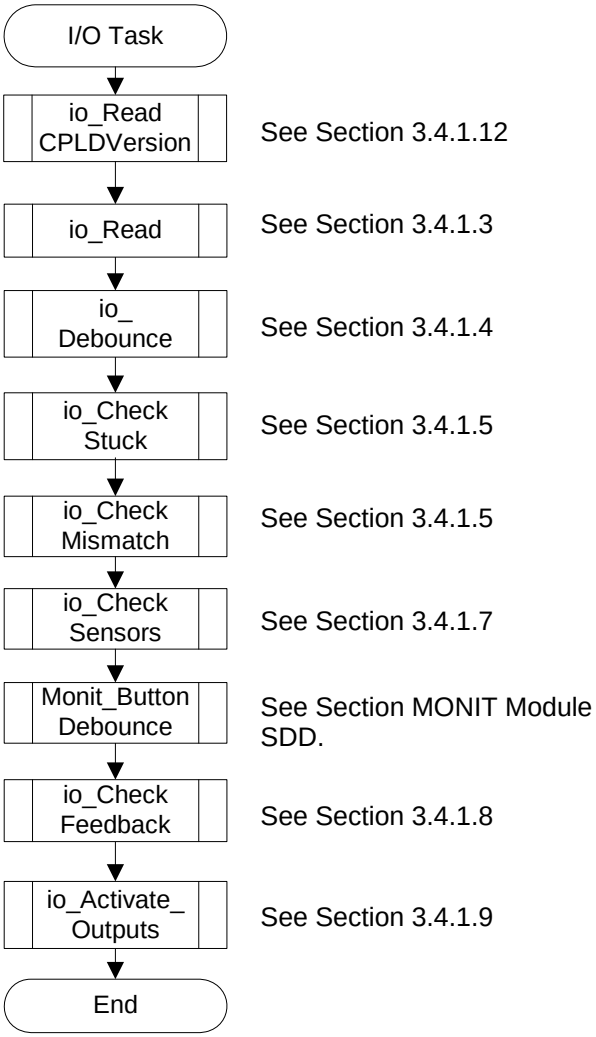

Figure 3-3: IO_Task Flowchart

3.4.1.3 io_Read Function

Figure 3-4 describes the process used to read all inputs.

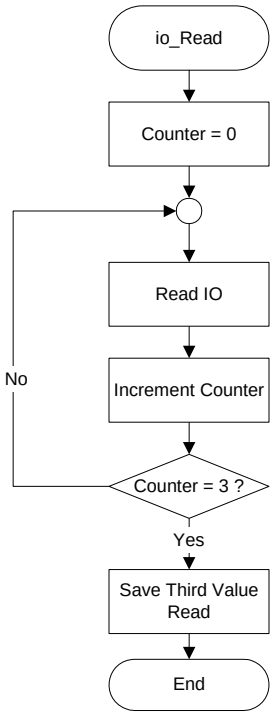

Figure 3-4: io_Read Flowchart

3.4.1.4 io_Debounce Function

Figure 3-5 describes the process used to debounce the inputs. Upon completion of debouncing, the DCU Application copies the local input structure into the global one.

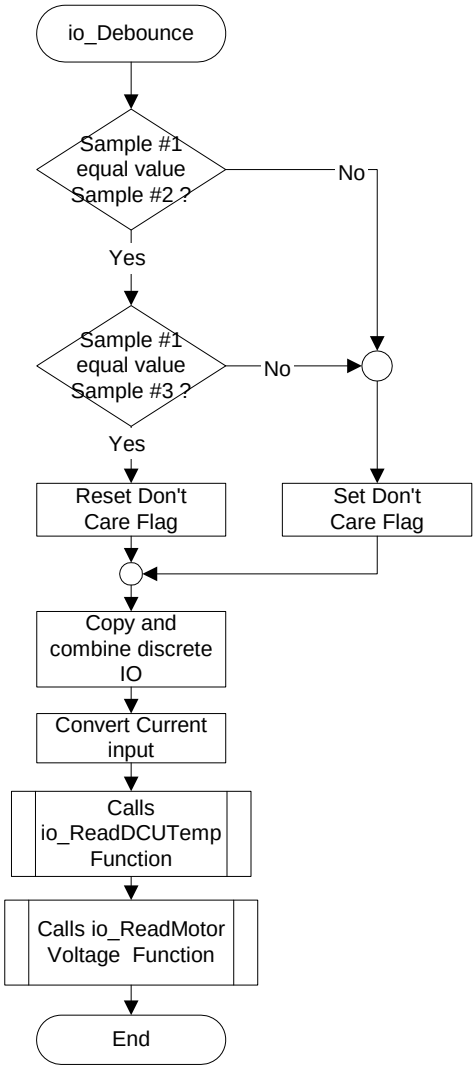

Figure 3-5: io_Debounce Flowchart

3.4.1.5 io_CheckStuck Function

This function is responsible for verifying the stuck high signals if they are active for more than 20 seconds. It is responsible for following events:

- Event 51, Door Open Power TL Stuck
- Event 52, Door Open Return TL Stuck
- Event 53, Door Open Crew Key Stuck
- Event 65, Door Close TL Stuck
- Event 66, Door Close Crew Switch Stuck
- Event 78, Passenger Door Enable TL Stuck
- Event 80, Passenger Door Open Pushbutton Stuck

3.4.1.6 io_CheckMismatch Function

This function is responsible for verifying mismatch between redundant signals. It is responsible for following events:

- Event 73, Zero Speed Carlines Mismatch, is set if two signals are different for 2 seconds.
- Event 21, PSS Mismatch, is set if two signals are different for 1 second.

Events will be reset when redundant signals are same.

3.4.1.7 io_CheckSensors Function

This function is used to set the global flags for the PSS, LSS, and the Open Interlock Status. The LSS flag is considered unlocked when any LSS is unlocked.

3.4.1.8 io_CheckFeedback Function

This function verifies if an over current occurred on one of the following outputs: Solenoid Lock Low Side, Solenoid High Side, Open Interlock Supply, Door Closing Visual Alarm, Door Closing Audible Alarm, Passenger Door Enable Light, and Spare Output.
In the io_CheckFeedback function, an over current is presumed when an output is set but there is no feedback of the output. That can be caused by an over current or a defective transistor. If no feedback is obtained after 50ms, the corresponding error is set. The event is reset only when the output is activated and the feedback is detected. These are the faults logged when over current occurs.

- Event 44, Door Closing Visual Alarm OC
- Event 46, Solenoid Low Side OC
- Event 48, Spare Output OC
- Event 81, Passenger Door Enable Light OC
- Event 88, Door Closing Audible Alarm OC
- Event 90, No Open Interlock Feedback with Output On
- Event 92, Solenoid High Side OC

If the feedback of an output is detected without the corresponding command, an event is set. The error is reset only when neither the output nor the feedback are activated. These are the faults logged when the output is being activated without command:

- Event 45, Door Closing Visual Alarm On W/O Command
- Event 47, Solenoid Low Side On W/O Command
- Event 49, Spare Output On W/O Command
- Event 82, Passenger Door Enable Light On W/O Command
- Event 89, Door Closing Audible On W/O o Command
- Event 91, Open Interlock Feedback On W/O Output On
- Event 93, Solenoid High Side On W/O Command

3.4.1.9 io_Activate_Outputs Function

This function drives outputs according to status in global output structure as seen in the Table 3-2.

3.4.1.10 io_ReadDCUTemp function

This function reads the DCU temperature using the analog input port AN1. The DCU board temperature is proportional to the voltage read with a slope of 1 °C/ 10 mV and a zero value of 500 mV. The expected range is between 100 mV and 2 V (40 °C to 150 °C). If the value read is outside this expected range, the Event 50, DCU Temperature Reading Error, is set. The Event 50 is reset the next time the DCU Temperature is within the expected limits.

3.4.1.11 io_ReadMotorVoltage Function

This function reads the Motor Voltage using the Timer 5. The voltage is calculated from the On time of a PWM input. The Voltage is obtained by finding the corresponding value in a look-up table. See Table 3-3 for the values. Two-timer values are retrieved: activation of the voltage input (t0) and deactivation of the voltage input (t1). The

On time is (t1 - t0). The Timer 5 is programmed to overflow if the maximum period is reached twice without transition on the input. If the Timer 5 overflows and the input pin is 0 or the ON time is over 45.41µS or under 9.54µS, the Event 33, Motor Voltage Reading Error, is set. The Event 33 is reset the next time the motor voltage input is read within expected limits.

Table 3-3: Motor Voltage Look Up Table

Volts	On time (µS)	Volts	On time (µS)	Volts	On time (µS)	Volts	On time (µS)
35	45.41	67	23.13	99	15.52	131	11.68
36	44.08	68	22.78	100	15.36	132	11.59
37	42.83	69	22.45	101	15.21	133	11.50
38	41.65	70	22.12	102	15.06	134	11.41
39	40.53	71	21.80	103	14.91	135	11.33
40	39.47	72	21.49	104	14.76	136	11.24
41	38.46	73	21.19	105	14.62	137	11.16
42	37.51	74	20.89	106	14.48	138	11.08
43	36.60	75	20.61	107	14.34	139	11.00
44	35.73	76	20.33	108	14.21	140	10.92
45	34.90	77	20.06	109	14.07	141	10.84
46	34.11	78	19.80	110	13.94	142	10.76
47	33.36	79	19.54	111	13.82	143	10.69
48	32.64	80	19.29	112	13.69	144	10.61
49	31.95	81	19.05	113	13.57	145	10.54
50	31.29	82	18.81	114	13.45	146	10.46
51	30.65	83	18.58	115	13.33	147	10.39
52	30.04	84	18.35	116	13.21	148	10.32
53	29.45	85	18.13	117	13.10	149	10.25
54	28.89	86	17.92	118	12.99	150	10.18
55	28.35	87	17.71	119	12.87	151	10.11
56	27.82	88	17.50	120	12.77	152	10.05
57	27.32	89	17.30	121	12.66	153	9.98
58	26.84	90	17.10	122	12.55	154	9.92
59	26.37	91	16.91	123	12.45	155	9.85
60	25.91	92	16.73	124	12.35	156	9.79
61	25.48	93	16.54	125	12.25	157	9.72
62	25.05	94	16.36	126	12.15	158	9.66
63	24.64	95	16.19	127	12.05	159	9.60
64	24.25	96	16.02	128	11.96	160	9.54
65	23.87	97	15.85	129	11.86		
66	23.49	98	15.68	130	11.77		

3.4.1.12 io_ReadCPLDVersion Function

The DCU sets Event 85, Invalid CPLD Version, if the version returned by the CPLD is null. The event is reset when the CPLD returns a version that is not null.
The DCU sets Event 86, CPLD Test Version, if the version returned by the CPLD indicates that it is not an official release. The event is reset when the CPLD returns an official release version.

3.4.1.13 IO_CurrentLevel Function

This function activates the normal current limit if the ucCommand is set to CURRENT_NORMAL. Otherwise it will set the normal current limit to off.

3.4.1.14 IO_Motor_Direction Function

This function activates corresponding transistors in the H bridge motor circuit according to the direction of the received command.

3.4.1.15 IO_ReadCurrent Function

The Read Current module reads the Motor Current using the analog input. The current is calculated from the duty cycle of a PWM input. 50% means 0 A; 7% means -13 A; and 93% means 13 A. Three timer values are retrieved: activation of the current input (t_0), deactivation of the current input (t_1), and the next activation of the current input (t_2). The duty cycle is (t_1 - t_0) / (t_2 - t_0). The Timer 5 is programmed to overflow if the maximum period is reached twice without transition on the input. If the Timer 5 overflows or the duty cycle is over 94% or under 6%, the Event 32, Motor Current Reading Error, is set. The Event 32 is reset the next time the current input is working within expected limits.

3.4.2 Data Entities Detailed Design

Not applicable.